Project Proposal: Real-Time Parallelized Image Segementation with K-means Clustering

Bert Shan, Charlotte Li

URL: https://github.com/BLS1202/15418-f25-Parallelized-K-means-Image-Segementation.git

Summary:

This project aims to accelerate the K-Means clustering algorithm for image segmentation by developing highly optimized parallel implementations for both multi-core CPUs and NVIDIA GPUs. We will implement the algorithm using OpenMP to leverage thread-level parallelism on CPUs and CUDA to exploit the massive data parallelism of GPUs. The primary goal is to compare the performance characteristics of these distinct parallel architectures and analyze the specific optimizations required to achieve significant speedup on each. We also want to have a program that takes an input image, and displays the process of image segmentation through CUDA rendering.

Background:

The K-Means algorithm partitions a set of data points into *K* distinct clusters. In the context of image segmentation, each pixel in the image is treated as a three-dimensional data point (representing its R, G, B color values). The algorithm iteratively refines the positions of *K* "centroids" (representative colors) to minimize the distance from each pixel to its assigned centroid, effectively segmenting the image into *K* color regions.

The algorithm proceeds in two main steps, repeated until convergence:

1. **Assignment Step:** For every pixel in the image, calculate the Euclidean distance to each of the *K* centroids. The pixel is then assigned to the cluster of its nearest centroid. This step is computationally expensive but "embarrassingly parallel," as the assignment for each pixel is completely independent of all other pixels. This data-parallel nature makes it a prime candidate for acceleration on both CPUs and GPUs.
2. **Update Step:** Recalculate the position of each of the *K* centroids by finding the mean color of all pixels assigned to its cluster. This step involves a parallel reduction, where the color values of all pixels within a cluster are summed up before being averaged.

The workload is defined by the image size (N pixels) and the number of clusters (K). The total computation per iteration is dominated by the Assignment step, involving N * K distance calculations, and the Update step, which requires iterating through all N pixels to perform reductions.

Challenge:

While the K-Means algorithm is highly data-parallel, achieving good speedup is challenging due to a significant synchronization bottleneck in the "Update Step." This step requires a many-to-one reduction where millions of pixel-processing threads must concurrently update the values of just *K* shared centroids. A naive implementation on both CPU and GPU architectures will suffer from severe performance degradation due to high-contention writes to these shared memory locations.

The core challenge is to design and implement efficient, architecture-specific reduction strategies:

- **On the GPU (CUDA):** A naive approach using global atomic operations would serialize threads, negating the benefit of massive parallelism. Furthermore, because pixels belonging to a cluster are scattered across the image, the memory access patterns for the reduction will be irregular and uncoalesced, leading to poor memory bandwidth utilization.
- **On the CPU (OpenMP):** A simple parallel loop protected by locks or atomic directives will cause high synchronization overhead and cache line contention, where multiple cores repeatedly invalidate each other's caches as they compete to update the same centroid data.

Our project will focus on solving these issues by implementing optimized reduction techniques, such as using shared memory on the GPU and per-thread private accumulators on the CPU, to minimize contention and maximize performance.

Resources

We will begin with the sequential version of k-means with starter code from here https://github.com/npav5057/K-Means-Clustering, note that this starter code is written in python, and we will implement the cpp version of this, and then work on the parallel optimization process.

We will also look at some other research papers:

https://dl.acm.org/doi/abs/10.1145/3404397.3404426

Goals:

75%: implement the cpp sequential version of image segmentation with k-means algorithm; and then optimize it with simple optimization with OpenMP

100%: implement the k-means algorithm of image segmentation with basic CUDA optimization

125%: CUDA optimization considering more aspects of memory bandwidth, cache limits, and workload dispatching by referencing the research paper: https://dl.acm.org/doi/abs/10.1145/3404397.3404426

Platform choice

We will use the GHC and PSC machines. We will use c++ as programming language, which will be helpful when optimizing with CUDA and OpenMP.

Schedule

11/12 – 11/17: choose topic and do literature review for the topic

11/17 – 11/24: implement the c++ sequential version of the algorithm, and get the basic optimization

11/24 – 12/1: optimize the parallel version and debug

12/1 – 12/8: further work on optimization and write report